

TÓM TẮT TIỂU LUẬN CÔNG NGHỆ PHẦN MỀM

Sinh viên thực hiện: Phan Trung Hiếu

MSSV: 2312616

Lớp: CTK47A

Người hướng dẫn: Gv. Lê Gia Công

1. TỔNG QUAN VỀ CÔNG NGHỆ PHẦN MỀM

Công nghệ phần mềm (Software Engineering) là ngành kỹ thuật bao quát toàn bộ quá trình sản xuất phần mềm, từ phân tích yêu cầu đến bảo trì. Không chỉ đơn thuần là lập trình, nó nhấn mạnh cách tiếp cận có hệ thống, có kỷ luật và có thể định lượng. Vòng đời phát triển phần mềm (SDLC) gồm 6 giai đoạn: Lấy và phân tích yêu cầu → Thiết kế → Lập trình → Kiểm thử → Triển khai → Vận hành & Bảo trì.

2. CÁC MÔ HÌNH PHÁT TRIỂN PHẦN MỀM

Mô hình	Đặc điểm	Phù hợp với
Thác nước	Tuần tự, tài liệu hóa cao	Yêu cầu ổn định, ít thay đổi
Chữ V	Kiểm thử song song với phát triển	Hệ thống độ tin cậy cao (y tế, hàng không)
Tiếp cận lặp	Chia thành nhiều vòng lặp, phản hồi liên tục	Dự án yêu cầu chưa rõ ràng
Agile/Scrum	Sprint 1–4 tuần, linh hoạt cao	Dự án thay đổi nhanh, startup

3. QUẢN LÝ DỰ ÁN PHẦN MỀM

- Dự án có 4 yếu tố ràng buộc PCTS: Performance (Chất lượng), Cost (Chi phí), Time (Thời gian), Scope (Phạm vi). Thay đổi một yếu tố sẽ ảnh hưởng đến các yếu tố còn lại.

- Nhà quản lý dự án đóng vai trò "enabler", không nên vừa quản lý vừa thực hiện dự án.

- Khoảng 83% dự án phần mềm gặp vấn đề về mục tiêu PCTS.

- Các công cụ hỗ trợ: WBS, biểu đồ Gantt, Jira, Trello.

4. LẤY VÀ PHÂN TÍCH YÊU CẦU

- **Yêu cầu chức năng:** Hệ thống làm gì (đăng nhập, tìm kiếm, thanh toán...).

- **Yêu cầu phi chức năng:** Hệ thống như thế nào (hiệu suất, bảo mật, khả năng mở rộng...).

- **Tài liệu URS** (User Requirement Specification): Góc nhìn người dùng, ngôn ngữ tự nhiên.

- **Tài liệu SRS** (Software Requirement Specification): Tài liệu kỹ thuật chi tiết, cơ sở cho thiết kế và kiểm thử.

5. THIẾT KẾ KIẾN TRÚC VÀ THIẾT KẾ CHI TIẾT

Các kiến trúc phần mềm:

Kiến trúc	Ưu điểm	Nhược điểm
Nguyên khối	Đơn giản, hiệu suất cao	Khó mở rộng, khó bảo trì
N – tầng (3 tầng)	Tái sử dụng, mở rộng độc lập	Phức tạp hơn nguyên khối
Vi dịch vụ	Độc lập, chịu lỗi tốt, đa dạng công nghệ	Phức tạp, chi phí vận hành cao

Thiết kế chi tiết gồm 4 nội dung: thiết kế dữ liệu, logic xử lý (mã giả, lưu đồ), API, phân cấp lớp. Sử dụng **Design Patterns**: Singleton, Adapter, Observer.

6. CÔNG CỤ HỖ TRỢ

Git – Quản lý phiên bản phân tán:

- **3 khu vực làm việc:** Working Directory → Staging Area → Repository.

- **Các lệnh cơ bản:** git init, git add, git commit, git push, git pull.

- **Cơ chế snapshot** (chụp ảnh) thay vì lưu delta.
- **Đảm bảo toàn vẹn dữ liệu** bằng mã SHA-1.
- Hỗ trợ làm việc nhóm qua phân nhánh (branch) và hợp nhất (merge).

Docker – Container hóa ứng dụng:

• **Quy trình Build – Ship – Run:** Viết Dockerfile → Build Image → Chạy Container.

- **Docker Image:** Khuôn mẫu bất biến chứa ứng dụng và môi trường.
- **Docker Container:** Thể hiện đang chạy của Image, biệt lập và nhẹ hơn VM.
- Giải quyết triệt để vấn đề "máy tôi chạy được, máy bạn thì không".

7. KẾT QUẢ BÀI TẬP VÀ THỰC HÀNH

• Hoàn thành bài tập: phân biệt CNPM và KHMT; vẽ sơ đồ 4 mô hình phát triển; viết URS, SRS cho hệ thống bán sách trực tuyến; thiết kế kiến trúc 3-tầng và Microservices.

• Thực hành Git: Khởi tạo repo, cấu hình local/global, quản lý trạng thái tập tin (Untracked, Staged, Committed), sử dụng .gitignore, xử lý sự cố với git rm --cached và git checkout.

• Thực hành Docker: Xây dựng Dockerfile (từ nginx:alpine), build Image hello-docker-image:v1, chạy Container và truy cập qua localhost:8080, xuất Image ra file .tar để chuyển giao.

8. KẾT LUẬN

Môn học Công nghệ phần mềm cung cấp nền tảng toàn diện về quy trình, phương pháp và công cụ phát triển phần mềm chuyên nghiệp. Em đã vận dụng kiến thức vào các bài tập phân tích, thiết kế và thực hành thành thạo Git, Docker – những công cụ thiết yếu trong ngành. Đây là hành trang quan trọng để tham gia vào các dự án thực tế với tư cách kỹ sư phần mềm chuyên nghiệp.